

Distributed Order-Processing System — Full Build Plan

A microservices order platform using the **Saga orchestration pattern**, built backend-first with a React frontend and deployed to a host. Stack: **Java 21, Spring Boot 3, Spring Kafka, Spring Data JPA, PostgreSQL, Redis, Docker, React**.

This is a build bible. Work through it phase by phase. Each phase is shippable on its own — at the end of every phase you have something that runs and demos, which is exactly what you want when you're building over weekends and might pause.

1. The system at a glance

A customer browses products, places an order, and the order is processed across four independent services that each own their own database. Because no single ACID transaction can span those databases, an **orchestrator** drives the steps and unwinds them with **compensating transactions** if any step fails.

Services

Service	Owns	Core responsibility
API Gateway	nothing (stateless)	Single entry point, routing, JWT validation, rate limiting
Order Service	orders, saga state	Order lifecycle + the saga orchestrator (the brain)
Inventory Service	products, stock	Product catalog + reserve/release stock
Payment Service	payments	Authorize / capture / refund (mock gateway)
Shipping Service	shipments	Schedule / cancel dispatch

Scope honesty: the four business services + gateway is the real project. Auth can start as a tiny piece inside the gateway and grow into its own service only if you want it. Don't add a Notification service, a Catalog service, etc. until the core works — scope creep is how weekend projects die unfinished.

How they talk

- **Frontend → Gateway:** synchronous REST/JSON.
- **Gateway → Order Service:** synchronous REST (the user is waiting for an order to be *accepted*).
- **Order Service ↔ business services:** asynchronous **Kafka** commands and events. No synchronous service-to-service calls on the saga path.

The happy path

```
Reserve inventory → Authorize payment → Schedule shipping → Confirm order
```

A failure path (payment fails at step 2)

```
Reserve inventory → Authorize payment ×  
    compensate: Release inventory → Cancel order
```

2. Repository & infrastructure layout

Use a **mono-repo with multiple Spring Boot modules** (simplest to build and run locally; mention in interviews you'd split to separate repos + a shared contracts library at real scale).

```
order-platform/  
├─ docker-compose.yml          # kafka, zookeeper, postgres(x4 or schemas),  
redis  
├─ gateway/                   # Spring Cloud Gateway  
├─ order-service/             # orchestrator lives here  
├─ inventory-service/  
├─ payment-service/  
├─ shipping-service/  
├─ common-events/             # shared event/command DTOs (a plain jar)  
└─ frontend/                  # React app
```

Database choice: one PostgreSQL instance, **one schema per service** (cheap locally, still enforces "no shared tables"). Say in interviews you'd use separate DB instances in production — the point is that no service reads another's tables.

PHASE 0 — Foundations (Weekend 1, first half)

Goal: infrastructure running, services can boot, they can publish/consume one Kafka message.

Deliverables:

- `docker-compose.yml` brings up Kafka + Zookeeper (or KRaft), PostgreSQL, Redis.
- Four empty Spring Boot services boot on different ports and expose `/actuator/health`.
- `common-events` module with a first event DTO.

- Smoke test: Order Service publishes a test message to a topic, Inventory Service logs it on consume.

Expectation at end: `(docker-compose up)` starts everything; you can prove one message flows Order → Kafka → Inventory.

PHASE 1 — Service skeletons with their own data (Weekend 1, second half)

Build each business service as a standalone CRUD app first. **No saga yet.** Get the data models and local logic solid before wiring orchestration — debugging a saga on top of half-built services is misery.

Inventory Service

- **Owns:** `(product)` (id, name, price, description), `(stock)` (product_id, available_qty, reserved_qty).
- **Local endpoints (for now):** `(GET /products)`, `(GET /products/{id})`, admin `(POST /products)`.
- **Expectation:** can list products and report stock. This is also what the frontend's product page will read.

Payment Service

- **Owns:** `(payment)` (id, order_id, amount, status, idempotency_key).
- **Local logic:** a mock gateway — approve if amount under a threshold, fail above it, so you can deterministically trigger failures later.
- **Expectation:** can record a payment and return APPROVED / DECLINED.

Shipping Service

- **Owns:** `(shipment)` (id, order_id, address, status).
- **Expectation:** can create a shipment record in SCHEDULED state.

Order Service

- **Owns:** `(order)` (id, user_id, items, total, status), `(order_item)`, and (added in Phase 3) `(saga_state)`.
- **Local endpoint:** `(POST /orders)` creates an order in `(CREATED)` status — does nothing downstream yet.
- **Expectation:** an order row gets created and persisted.

Expectation at end of Phase 1: every service works in isolation, has its own tables, and you can hit each with Postman.

PHASE 2 — Happy-path saga orchestration (Weekend 2)

Goal: `POST /orders` drives all four steps to completion over Kafka, end to end, when nothing fails.

Define the contracts (`common-events`)

Commands (orchestrator → service) and events (service → orchestrator):

```
ReserveInventoryCommand    → InventoryReservedEvent /  
InventoryReservationFailedEvent  
AuthorizePaymentCommand    → PaymentAuthorizedEvent / PaymentFailedEvent  
ScheduleShippingCommand    → ShippingScheduledEvent / ShippingFailedEvent
```

Every message carries a `sagaId` (correlation id) so everything can be traced and deduped.

Build the orchestrator (in Order Service)

On `POST /orders`: persist the order, start a saga, publish `ReserveInventoryCommand`, return **202 Accepted** to the user immediately (the order is *accepted*, not yet *completed* — this is the two-phase idea you already understand from UCC). The orchestrator then reacts to each event and fires the next command.

Each business service: consume its command, do the work, publish success or failure event.

Expectation at end of Phase 2: placing an order moves it through `CREATED → INVENTORY_RESERVED → PAYMENT_COMPLETED → SHIPPED → COMPLETED`, all driven by Kafka, with no failures handled yet. You can watch the status change by polling `GET /orders/{id}`.

PHASE 3 — Failure handling & compensation (Weekend 3 — the core of the project)

Goal: when a step fails, the saga unwinds everything before it. This is the phase that makes the project interview-worthy. Spend the most time here.

Persist saga state (do this first)

Add a `saga_state` table: `saga_id`, `order_id`, `current_step`, `status`, `created_at`, `updated_at`. The orchestrator reads/writes this on every transition so it can **recover after a crash** — on restart it knows "I was awaiting shipping confirmation for saga X."

Model the lifecycle as a state machine:

```
STARTED → INVENTORY_RESERVED → PAYMENT_AUTHORIZED → SHIPPING_SCHEDULED →  
COMPLETED  
    ↳ any *_FAILED → COMPENSATING → COMPENSATED (order CANCELLED)
```

Define compensating commands

ReleaseInventoryCommand	(undoes ReserveInventory)
RefundPaymentCommand	(undoes AuthorizePayment)
CancelShippingCommand	(undoes ScheduleShipping)

Wire the unwind

On any failure event, the orchestrator transitions to `COMPENSATING` and fires the inverse commands **for steps already completed, in reverse order**, then marks the order `CANCELLED`.

Add timeouts

If a service never responds, a per-step timeout (scheduled check against `saga_state`) triggers compensation so a saga can't hang forever.

Expectation at end of Phase 3: force a payment failure (order total above the threshold) and watch inventory get released and the order cancelled automatically. Kill a service mid-saga and show the timeout firing. **Screenshot these** — they become your strongest interview stories.

PHASE 4 — Reliability hardening (Weekend 4)

Goal: make it correct under the messy realities of distributed messaging.

- **Idempotent consumers:** Kafka is at-least-once, so every consumer must dedupe on `sagaId` + message type (store processed ids, or use a unique constraint). Without this, a redelivered "reserve inventory" double-decrements stock.
- **Transactional Outbox:** in the Order Service, write the outgoing event to an `outbox` table in the *same* DB transaction as the saga-state change; a relay/poller publishes to Kafka. This fixes the dual-write problem (DB commits but Kafka publish fails).
- **Circuit breakers & retries:** Resilience4j on the mock external payment call; retry transient failures with backoff before declaring a step failed.
- **Concurrency control:** optimistic locking (`@Version`) on `stock` so two simultaneous orders for the last unit can't both succeed.

Expectation at end of Phase 4: redelivering the same Kafka message changes nothing; a publish failure doesn't lose an event; concurrent orders for scarce stock are handled correctly.

PHASE 5 — API Gateway & auth (Weekend 5, first half)

Goal: one front door, secured.

- **Spring Cloud Gateway** as the single entry point; route `/api/orders/**`, `/api/products/**` to the right services. Frontend only ever talks to the gateway.
- **JWT auth:** start minimal — a `/auth/login` that issues a JWT (a small user store; bcrypt passwords). Gateway validates the JWT on protected routes and forwards user identity downstream.
- **Rate limiting** at the gateway (Redis-backed) — ties back to concepts you already know and is a nice talking point.

Expectation at end of Phase 5: the frontend hits a single base URL, must log in to place an order, and unauthenticated requests are rejected at the edge.

PHASE 6 — Frontend (Weekend 5, second half → Weekend 6)

Goal: a real React app — this is what makes "Full Stack" honest on your resume and what makes the hosted demo clickable.

Stack: React + Vite, React Router, Axios, Tailwind (or plain CSS). Keep it clean, not fancy.

Pages/flows:

1. **Login / register** — stores JWT, attaches it to every request.
2. **Product listing** — reads from Inventory via the gateway; shows stock.
3. **Cart** — client-side state; add/remove items.
4. **Checkout** — `POST /api/orders`, receives **202 Accepted**.
5. **Order status / saga progress** — the standout UI: poll `GET /orders/{id}` and render the saga as a live stepper (Inventory ✓ → Payment ✓ → Shipping ...), including the **compensation/cancelled** path when something fails. This visualizes the distributed flow and is genuinely impressive in a demo.
6. **Order history** — list past orders.

Expectation at end of Phase 6: a logged-in user can browse, add to cart, check out, and *watch* their order progress (or fail and unwind) through the saga in real time.

The order-status stepper is worth extra polish. Most candidates' projects have no UI at all; a screen that animates a distributed saga — including the failure-and-rollback case — is a memorable demo.

PHASE 7 — Observability (Weekend 6 or folded in earlier)

Goal: be able to answer "a saga is stuck — how do you find it?"

- **Correlation IDs:** propagate `sagaId` / a trace id through every log line across services (MDC in logback). Now one order's whole journey is greppable.
- **Structured logging** at each transition.
- **Metrics:** Spring Boot Actuator + Micrometer; optionally Prometheus + Grafana in docker-compose for orders placed, sagas compensated, step latencies.

Expectation at end of Phase 7: you can trace a single order across all four services by its id, and you have basic dashboards/counters. This alone puts you ahead of most candidates, who have nothing here.

PHASE 8 — Containerization & hosting (final)

Goal: the app is live at a URL you can put in a README and demo on the spot.

- **Dockerize every service** (multi-stage builds: build the jar, copy into a slim JRE image). Frontend builds to static files served by Nginx or the gateway.
- `docker-compose.prod.yml` wiring all services + Kafka + Postgres + Redis with prod-ish config and env vars (no secrets in the repo).
- **Hosting options (cheapest realistic first):**
 - A single **VPS** (Hetzner / DigitalOcean / Oracle Cloud free tier / an AWS EC2 instance) running `docker-compose up`. Simplest for a full multi-service stack on a budget. **Recommended start.**
 - **AWS ECS / a managed container service** if you want to show cloud-native deployment (more setup, stronger talking point, ties to the AWS on your resume).
 - Kubernetes only if you specifically want the k8s story — otherwise it's overkill for a demo and a time sink.
- **Domain + HTTPS:** a cheap domain, Nginx reverse proxy, Let's Encrypt TLS.
- **CI/CD (optional but high-value):** a GitHub Actions workflow that builds images and deploys on push — directly reinforces the "CI/CD" claim on your resume.

Expectation at end of Phase 8: a public URL, a README with an architecture diagram and a "how it works" section, and a one-command local spin-up. The README is what a recruiter actually skims — make it good.

3. Per-service expectations summary

Order Service — the brain. Owns order lifecycle + saga orchestration + saga-state persistence + Outbox. Must: accept orders (202), drive the saga, compensate on failure, recover after crash, expose order status. This service is where most of your interview answers come from.

Inventory Service — owns products and stock. Must: serve the catalog (read path for frontend), reserve stock idempotently, release on compensation, handle concurrent reservations via optimistic locking.

Payment Service — owns payments. Must: authorize/capture idempotently, refund on compensation, deterministically fail above a threshold (your failure-injection lever), survive redelivered commands.

Shipping Service — owns shipments. Must: schedule, cancel on compensation, idempotent. The simplest service — good one to build first to learn the command/event rhythm.

API Gateway — owns nothing. Must: route, validate JWT, rate-limit, be the only public surface.

4. Build order recap (so you never wonder "what's next")

1. **P0** Infra up, one message flows.
2. **P1** Each service works alone with its own DB.
3. **P2** Happy-path saga end to end.
4. **P3** Compensation + saga state + timeouts. ← the heart
5. **P4** Idempotency + Outbox + circuit breakers + locking.
6. **P5** Gateway + JWT auth.
7. **P6** React frontend with live saga stepper.
8. **P7** Correlation IDs + metrics.
9. **P8** Dockerize + host + README.

Each phase ends in a working demo. If you have to stop after Phase 3, you still have a genuinely strong project; everything after raises the ceiling.

5. A note on what actually wins the interview

The phases give you a working system. What makes it *stand out* is depth in Phase 3 and Phase 4 — being able to walk through a compensation firing, explain orchestrator crash recovery, and name the dual-write problem cold. Build those parts for real, break them on purpose, and keep notes on what went wrong. "Here's the bug I hit when a redelivered message double-reserved stock, and here's how I fixed it" is the single most convincing thing you can say, and it only exists if you actually built it.